



SQL DML e o **SELECT** Simples

Aula 14 · Bloco 4 — SQL Básico

Pôr dado dentro, tirar dado de dentro — e a armadilha do vazio

Nesta aula

1. `INSERT` — sempre com a **lista de colunas**
2. `UPDATE` e o `WHERE` que faltou
3. `DELETE`
4. `SELECT`, `WHERE`, `ORDER BY`
5. `IS NULL`, `LIKE`, `BETWEEN`, `IN`
6. `DISTINCT` e `LIMIT`

INSERT

```
INSERT INTO usuario (matricula, nome, email, categoria)
VALUES ('202500100', 'Joana Ribeiro', 'joana.ribeiro@escola.br', 'aluno');
```

```
INSERT INTO area (codigo_area, nome) VALUES
    ('IA', 'Inteligência Artificial'),
    ('SEG', 'Segurança da Informação');
```

`data_cadastro` não foi informada — o banco põe o `DEFAULT` que você declarou.

A lista de colunas é opcional em SQL e obrigatória aqui

Sem ela, os valores são **posicionais** —
e a ordem das colunas é justamente
o que o modelo relacional diz **não** existir.

Um **ALTER TABLE** de alguém
quebra o seu **INSERT** em silêncio,
gravando e-mail na coluna de categoria.

UPDATE

```
UPDATE emprestimo
  SET data_devolucao = DATE '2026-03-14',
      data_prevista  = DATE '2026-03-16'
 WHERE id_emprestimo = 1;
```

⚠ **UPDATE** sem **WHERE** altera a tabela inteira. Não há confirmação, não há aviso, e a resposta **UPDATE 4127** chega depois.

A regra que vale para o resto da carreira

Escreva o comando primeiro como **SELECT** .

Confira o número de linhas.

Só então troque **SELECT *** por **UPDATE ... SET** .

```
BEGIN;  
UPDATE ...;  
-- confira; se estiver errado: ROLLBACK;  
COMMIT;
```

SELECT , WHERE , ORDER BY

```
SELECT titulo, ano_publicacao, editora
FROM obra
WHERE ano_publicacao >= 2011
ORDER BY ano_publicacao DESC;
```

titulo	ano_publicacao	editora
Engenharia de Software	2018	Pearson
Introdução à Estatística	2015	LTC
Redes de Computadores	2011	Pearson

⚠ Sem **ORDER BY**, não há ordem garantida

É a propriedade da Aula 01
aparecendo na prática.

Se o resultado **parece** ordenado,
é coincidência do plano de execução —
e ela muda quando a tabela cresce.

IS NULL — o mais importante da aula

```
SELECT id_emprestimo, matricula, data_prevista  
FROM emprestimo  
WHERE data_devolucao IS NULL;      -- ✅ 4 empréstimos em aberto
```

```
SELECT id_emprestimo FROM emprestimo WHERE data_devolucao = NULL;
```

```
id_emprestimo  
-----  
(0 rows)
```

Zero linhas, sem erro e sem aviso

A comparação com vazio
não deu falso — deu **desconhecido**.

E o filtro só aceita
o que é **verdadeiro**.

É a armadilha mais cara do SQL básico,
porque não parece um erro:
parece que não há empréstimos em aberto.

Os outros três operadores

Operador	Para quê	Exemplo
LIKE	texto que contém, começa ou termina	titulo LIKE '%Software%'
BETWEEN	faixa, inclusive as pontas	ano BETWEEN 2010 AND 2015
IN	um valor de uma lista	categoria IN ('professor','servidor')

No **LIKE**: **%** casa com qualquer coisa, **_** casa com um caractere.

DISTINCT e LIMIT

```
SELECT DISTINCT editora FROM obra ORDER BY editora;
```

```
SELECT tombo, data_aquisicao FROM exemplar  
ORDER BY data_aquisicao LIMIT 3;
```

DISTINCT existir é a lembrança de que o SQL, ao contrário da teoria, **aceita repetição**.

💡 **LIMIT** sem **ORDER BY** devolve "três linhas quaisquer", não "as três primeiras" — não existem "primeiras" sem ordem declarada.

Exercícios da aula

Na pasta `aula-14/` :

1. `ex01.sql` — a carga do **seu** modelo, 5 linhas por tabela, na ordem certa;
2. `ex02.sql` — devolução e situação do exemplar, dentro de **uma transação**;
3. `ex03.sql` — cinco consultas com filtro, com os resultados colados;
4. `ex04.sql` — quatro perguntas sobre nulos, inclusive por que `= NULL` falha;
5. **Desafio** 🌶️ `ex05.sql` — reconstrua o estrago de um `UPDATE` sem `WHERE` .

Próxima aula

Aula 15 — Junções e agregação

A normalização separou.
Agora a tela do atendente
precisa dos dois juntos.